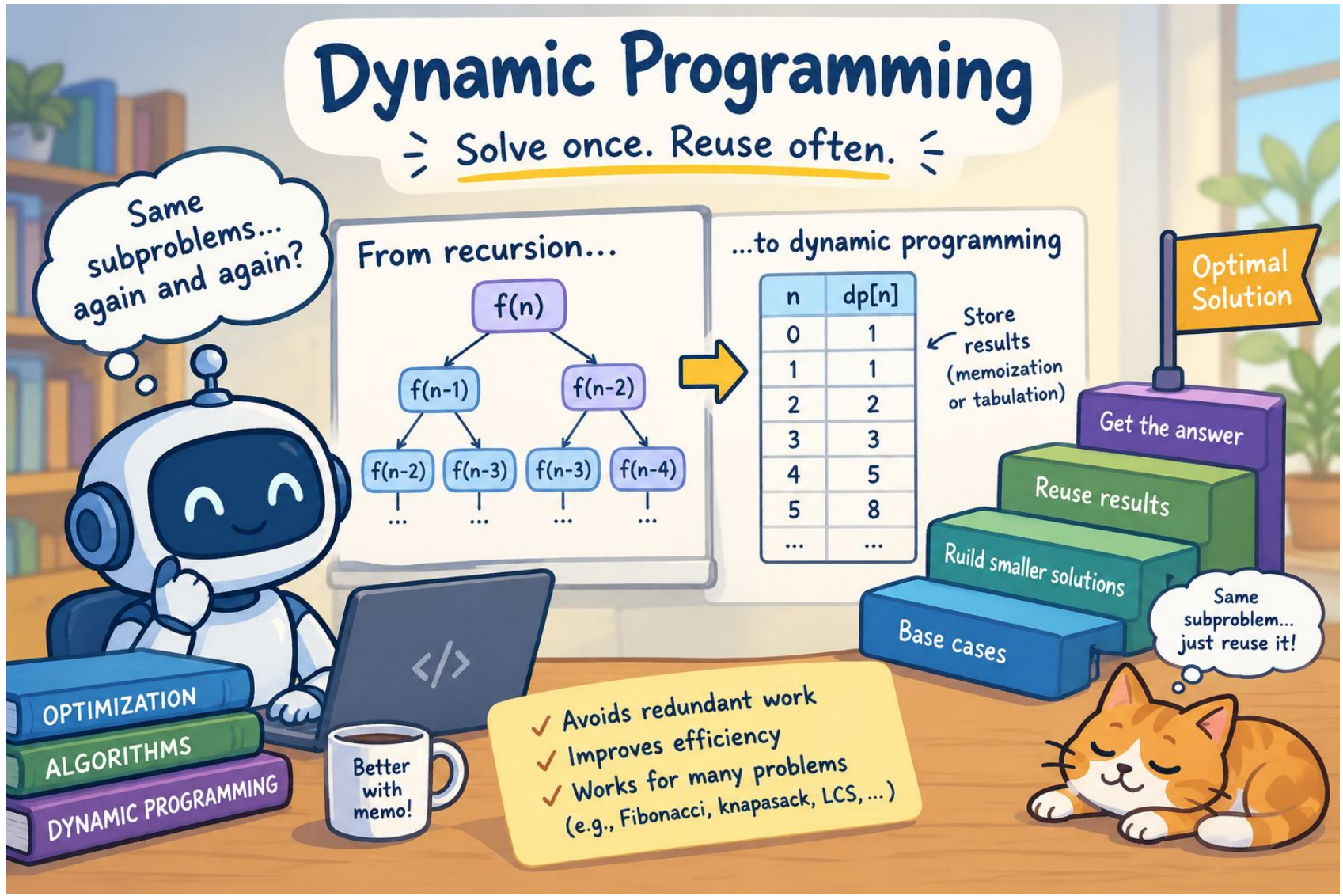
Algorithms & Data Structures

Week 05

Ahmet Feyzi Ateş

Dynamic Programming



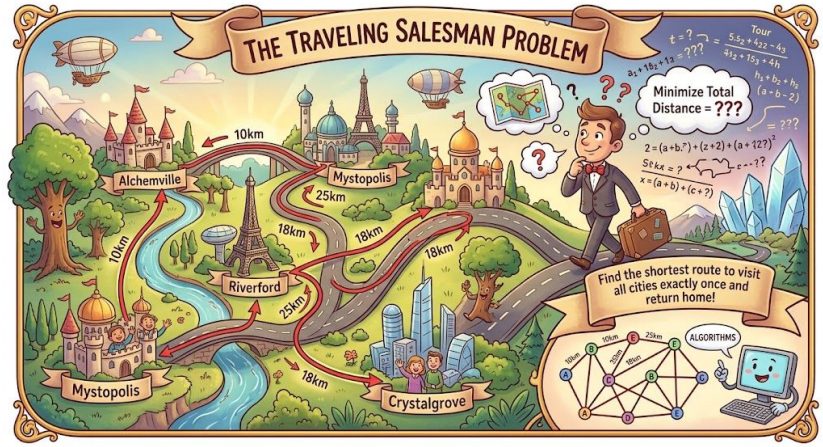
Exhaustive Search

A brute force solution to a problem involving search for an element with a special property, usually among combinatorial objects such as permutations, combinations, or subsets of a set.

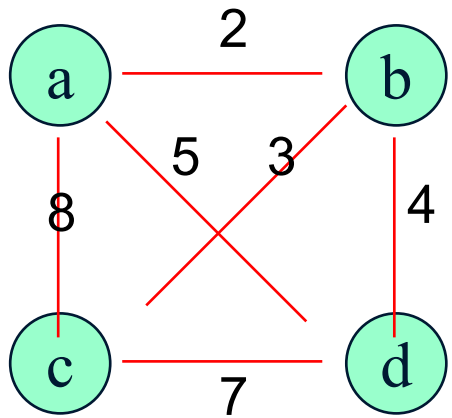
Method:

- generate a list of all potential solutions to the problem in a systematic manner
- evaluate potential solutions one by one, disqualifying infeasible ones and, for an optimization problem, keeping track of the best one found so far
- when search ends, announce the solution(s) found

Example 1:
Traveling Salesman Problem



- Given *n* cities with known distances between each pair,
  - find the **shortest** tour that passes through **all the cities** exactly **once** before returning to the **starting city**
- Alternatively: Find shortest *Hamiltonian circuit* in a weighted connected graph
- Example:



TSP by Exhaustive Search

Tour	Cost
a→b→c→d→a	2+3+7+5 = 17
a→b→d→c→a	2+4+7+8 = 21
a→c→b→d→a	8+3+4+5 = 20
a→c→d→b→a	8+7+4+2 = 21
a→d→b→c→a	5+4+3+8 = 20
a→d→c→b→a	5+7+3+2 = 17
b→a→c→d→b	2+8+7+4 = 21
b→a→d→c→b	
……	

Complexity: ?

Example 2: Knapsack Problem

Given *n* items:

- **weights:** w_1 w_2 … w_n
- **values:** v_1 v_2 … v_n
- **a knapsack of capacity *W***

Find most valuable subset of the items that fit into the knapsack

Example: Knapsack capacity W=16

item	weight	value
1	2	$20
2	5	$30
3	10	$50
4	5	$10



Knapsack Problem by Exhaustive Search

Subset	Total weight	Total value
{1}	2	$20
{2}	5	$30
{3}	10	$50
{4}	5	$10
{1,2}	7	$50
{1,3}	12	$70
{1,4}	7	$30
{2,3}	15	$80
{2,4}	10	$40
{3,4}	15	$60
{1,2,3}	17	not feasible
{1,2,4}	12	$60
{1,3,4}	17	not feasible
{2,3,4}	20	not feasible
{1,2,3,4}	22	not feasible

Complexity :

T(n) is $O(2^n)$

Example 3: The Assignment Problem

There are n people who need to be assigned to n jobs, one person per job. The cost of assigning person i to job j is C[i,j]. Find an assignment that minimizes the total cost.

	Job 0	Job 1	Job 2	Job 3
Person 0	9	2	7	8
Person 1	6	4	3	7
Person 2	5	8	1	8
Person 3	7	6	9	4

Algorithmic Plan: Generate all legitimate assignments, compute their costs, and select the cheapest one.

How many assignments are there?

Assignment Problem by Exhaustive Search

$$C = \begin{matrix} 9 & 2 & 7 & 8 \\ 6 & 4 & 3 & 7 \\ 5 & 8 & 1 & 8 \\ 7 & 6 & 9 & 4 \end{matrix}$$

Assignment (col.#s)	Total Cost
1, 2, 3, 4	9+4+1+4=18
1, 2, 4, 3	9+4+8+9=30
1, 3, 2, 4	9+3+8+4=24
1, 3, 4, 2	9+3+8+6=26
1, 4, 2, 3	9+7+8+9=33
1, 4, 3, 2	9+7+1+6=23

etc.

Fibonacci numbers

The Fibonacci numbers:

0, 1, 1, 2, 3, 5, 8, 13, 21, …

The Fibonacci numbers are a sequence F n of integers in which every number after the first two, 0 and 1, is the sum of the two preceding numbers. Hence the sequence:

0, 1, 1, 2, 3, 5, 8, 13, 21, …

Formally, they are defined by the Fibonacci recurrence:

$F(n) = F(n\text{-}1) + F(n\text{-}2)$

$F(0) = 0$

$F(1) = 1$

Fibonacci numbers

ALGORITHM *Fibonacci*(n)

//Computes the nth Fibonacci number recursively by definition
//**Input**: A nonnegative integer n
//**Output**: The nth Fibonacci number.
if $n = 0$ then
    return 0
else if n = 1 then
    return 1
else
    return *Fibonacci*(n-1) + *Fibonacci*(n-2)

F (n) = F(n−1) + F(n−2)

= F(n−2) + F(n−3) + F(n−3) + F(n−4)

= F(n−3) + F(n−4) + F(n−4) + F(n−5)

F(n−4) + F(n−5) + F(n−5) + F(n−6)

= F(n−4) + F(n−5) + F(n−5) + F(n−6) + F(n−5) + F(n−6) + F(n−6) + F(n−7) +

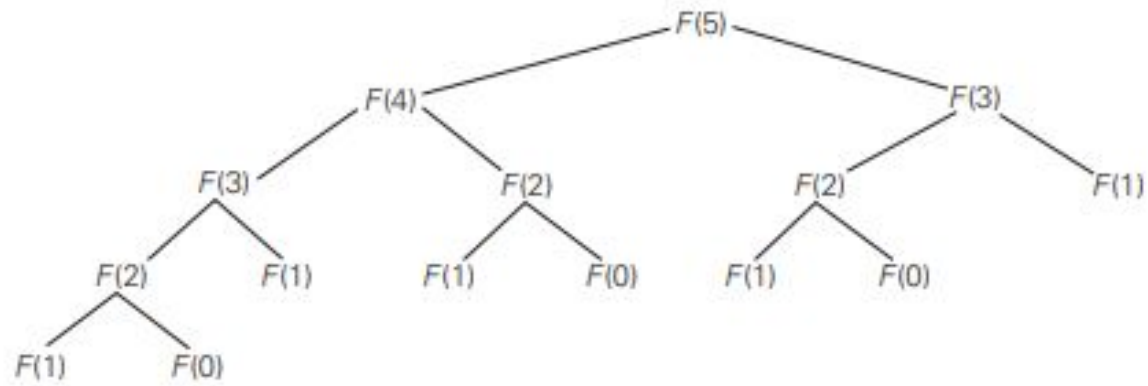
F(n−5) + F(n−6) + F(n−6) + F(n−7) + F(n−6) + F(n−7) + F(n−7) + F(n−8)

Quickly leads to exponential complexity!

Why exponential time complexity

- The exponential efficiency class of the algorithm's time complexity could be anticipated by the nature of recurrence.
- It contains two recursive calls with the sizes of smaller instances only slightly smaller than size n.
- We can also see the reason behind the algorithm's efficiency by looking at a recursive tree of calls tracing the algorithm's execution.

An example of call tree for n = 5



$$T(n) \in \Theta(\varphi^n)$$

Fibonacci numbers (cont.)

Computing the nth Fibonacci number using bottom-up iteration and recording results:

$F(0) = 0$
$F(1) = 1$
$F(2) = 1+0 = 1$
…
$F(n\text{-}2) =$
$F(n\text{-}1) =$
$F(n) = F(n\text{-}1) + F(n\text{-}2)$

0	**1**	**1**	**. . .**	***F*(*n*-2)**	***F*(*n*-1)**	***F*(*n*)**

Complexity:
- time
- space

Much Faster Iterative implementation

ALGORITHM ***Fibonacci_iterative*****(n)**

```
//Computes the n^th Fibonacci number iteratively
//Input: A nonnegative integer n
//Output: The n^th Fibonacci number.
F[0] ← 0; F[1] ←1
for i ← 2 to n do
    F[i] ←F[i-1] + F[i-2]
return F[n]
```

- Time complexity:
  - If problem size is n:
    - *O(n) i.e. linear with n*
- Space complexity:
    - *O(n) i.e. linear with n*

Dynamic Programming

D*ynamic Programming* is a general algorithm design technique for solving problems defined by recurrences with overlapping subproblems.

Related to the *space-for-time trade-off* idea. It is Based on recording solutions to overlapping subproblems of a given problem in a table from which a solution to the original problem is then obtained.

Invented by American mathematician Richard Bellman in the 1950s to solve optimization problems and later assimilated by CS.

"Programming" here means "planning".

Main idea:

- set up a recurrence relating a solution to a larger instance to solutions of some smaller instances
- solve smaller instances once
- **record** the solutions in a table
- extract solution to the initial instance from that table.

Example 1: Coin-row problem

There is a row of n coins whose values are some positive integers c_1, c_2,...,c_n, not necessarily distinct. The goal is to pick up the maximum amount of money subject to the constraint that no two coins adjacent in the initial row can be picked up.

E.g.: 5, 1, 2, 10, 6, 2. What is the best selection?

Algorithm *coinRowRecursive*(C[1..n])
//**Input** : Array C[1..n] of positive integers indicating the coin values
//**Otput**: The maximum amount of money that can be picked up, subject to the constraint that no two neighbour coins can be selected.
// Handle the base cases.
if n=0
return 0
else if n=1
return C[1]
else
return *max*(C[n] + *coinRowRecursive*(C[1..n-2]), *coinRowRecursive*(C[1..n-1]))

Time Complexity of the Solution

We can consider the basic operation of the above algorithm as determining the maximum of two values.

Since the maximum of two values can be found with a single comparison operation, it is obvious that this operation can be done in constant time.

If T(n) is the number of max operations that our algorithm will execute to find the value of F(n), then :

- T(n-1) will represent the value of F(n-1), and
- T(n-2) will represent the number of max operations that our algorithm will run to calculate the value of F(n-2).

Finally, our algorithm performs one more max operation to reach the result.

Therefore, we obtain the following "recurrence" relationship for T(n).

$$T(n) = T(n-1) + T(n-2) + 1 \quad \text{for } n>1$$
$$T(1) = 1$$
$$T(0) = 1$$

$$T(n) \in \Theta(\varphi^n)$$

DP solution to the coin-row problem

Let F(n) be the maximum amount that can be picked up from the row of n coins. To derive a recurrence for F(n), we partition all the allowed coin selections into two groups:

those without last coin – the max amount is ?

those with the last coin -- the max amount is ?

Thus we have the following recurrence

$$F(n) = \max\{c_n + F(n\text{-}2),\ F(n\text{-}1)\} \quad \text{for } n > 1,$$

$$F(0) = 0,\ F(1)=c_1$$

DP solution to the coin-row problem (cont.)

$$F(n) = \max\{c_n + F(n\text{-}2),\ F(n\text{-}1)\} \quad \text{for } n > 1,$$

$$F(0) = 0,\ F(1) = c_1$$

index	0	1	2	3	4	5	6
coins	--	5	1	2	10	6	2
F( )		5	5	7	15	15	17

Max amount: 17
Coins of optimal solution: 1, 4, 6
Time complexity: O(n)
Space compexity: O(n)

Note: All smaller instances were solved.

Pseudocode for Coin-Row with DP

Algorithm *coinRow(C[1..n])*

// **Input:** An array of n positive integers representing the values of the n coins in a row.

// **Output:** The maximum value of total money that can be taken from the row, provided that no two neighboring coins are taken.

F[0] ← 0

F[1] ← C[1]

for i ← 2 **to** n **do**

    F[i] ← *max*(C[i] + F[i – 2], F[i – 1])

return F[n]

Algorithm *max(a, b)*

// **Input:** Tow comparable values

// **Output:** The largest of the two

if a > b **then**

    return a

else

    return b

Example 2: Change-making problem

During a purchase, we will give n value (*n TL or $n*) change in the form of coins. We have m coins printed by our central bank, each with a different value;

$$d_1 < d_2 < d_3 \ldots\ldots\ldots < d_m$$

The goal is to give back the change to the buyer with the least number of coins possible .

We can assume that we have an infinite number of coins of each value *d*.

Additionally: $d_1 = 1$

Solution to the change-making problem with DP

Let F(n) represent the minimum number of coins whose monetary value sums to n.

We can accept F(0) = 0.

The only way we can arrive at the value of n is by adding one d_j coin (j = 1, 2, ….., m) to the total value ($n - d_j$);
provided that $n >= d_j$

So, we should consider all the possibilities (j = 1, 2, ….., m) and choose the one that minimizes $F(n-d_j)+1$.

Since 1 is a constant value, we can simply find $F(n-d_j)$ using minimum coins and add 1 to it.

Solution to the change-making problem with DP

This brings us to the following recurrence relation:

$$F(n) = \min_{j:\, n >= d_j} \{F(n-d_j)\} + 1 \quad \text{if } n > 0,$$

$$F(0) = 0.$$

We can calculate F(n) by filling in a table of rows from left to right, just as in the coin-row problem.

Different than the coin-row, here we will need to find the minimum among (at most) m numbers to find the value to be written into one cell of the table.

Solution to the change-making problem with DP

We have; n = 6, m = 3 and,

d_1=1, d_2=3, d_3= 4

F(0) = 0

n	0	1	2	3	4	5	6
F	**0**						

F(1)=min{F(1-1)}+1 =1

n	0	1	2	3	4	5
F	**0**	**1**				

F(2)=min{F(2-1)}+1 =2

n	0	1	2	3	4	5
F	**0**	**1**	**2**			

Solution to the change-making problem with DP

F(3)=min{F(3-1),F(3-3)}
+1
=1

n	0	1	2	3	4	5	6
F	**0**	**1**	**2**	**1**			

F(4)=min{F(4-1),F(4-3),
F(4-4)}+1
=1

n	0	1	2	3	4	5	6
F	**0**	**1**	**2**	**1**	**1**		

F(5)=min{F(5-1),F(5-3),
F(5-4)}+1
=2

n	0	1	2	3	4	5	6
F	**0**	**1**	**2**	**1**	**1**	2	

F(6)=min{F(6-1),F(6-3),
F(6-4)}+1
=2

n	0	1	2	3	4	5	6
F	**0**	**1**	**2**	**1**	**2**	**2**	**2**

Pseudocode for Change-making with DP

Algorithm ***ChangeMaking(D[1..m], n)***

// **Input:** An array of m elements representing the coin values at hand, consisting of positive integers of increasing value, where D[1] = 1, and a positive integer n.

// **Output:** The minimum ***number*** of coins whose values sum to n.

```
F[0] ← 0
for i ← 1 to n do
    temp ← MAXINT
    j ← 1
    while j <= m and i >= D[j] do
        temp ← min(F[i – D[j]], temp)
        j ← j + 1
    F[i] ← temp + 1
return F[n]
```

Algorithm *min(a, b)*

// **Input:** Tow comparable values

// **Output:** The largest of the two

```
if a < b then
    return a
else
    return b
```

Complexity analysis of Change-making with DP

Min coin count: 2

The coins: {3, 3} (We used d_3 for F(6), and also used d_3 for F(3) as well).

Worst-case time Complexity: O(n.m)

Best-case time Complexity : O(n) $[d_1=1, d_2>>n, d_3>>n, \ldots d_m>>m]$

Space Complexity: O(n)

Solution to the Knapsack problem with DP

If we limit our solution to items with integer weights and values, and knapsacks with integer capacities; then smaller (sub)problems can be defined as:

Let a total of n (sub)problems be :

$1 \le i \le n$ items with weights $\mathbf{w_1, w_2, w_3, ...w_n}$ and values $\mathbf{v_1, v_2, v_3, \ldots v_n}$
and
$1 \le j \le W$ be the knapsack (bag) capacities.

Let *F(i, j)* represent the optimum solution of the problem for a knapsack with i items and capacity j,

We can divide all subsets of the first i items into two groups:

- subsets that contain the ith item, and
- subsets that do not contain the ith item *F(i-1, j)*.

Solution to the Knapsack problem with DP

Among the subsets that do contain the ith item (which also means that $(j - w_i) \geq 0$, i.e. if the ith item is part of the solution, it must fit into a bag of capacity j, otherwise it would not be possible for the ith item to be part of the solution), an optimal subset can be obtained by adding this i^{th} item to the optimal subset formed by the first *(i -1)* items that can fit into a bag of capacity $(j - w_i)$. The total value of such an optimal subset will be :

$$v_i + F(i-1, j-w_i).$$

Henceforth, the optimal solution among the possible subsets of the first *i* items must be the one that is larger than the two values $(F(i-1, j), v_i + F(i-1, j-w_i))$ given above.

For the case where the ith item does not fit into the bag of capacity *j*, the optimal value sought for will be equal to the total value of the optimal subset obtained among the first *i-1* items, i.e., *F(i-1, j),* as noted above.

Solution to the Knapsack problem with DP

These observations lead us to the following “recurrence” relation.

$$V[i, j] = \max (V[i-1,j], v_i + V[i-1,j- w_i]) \quad \text{if } j-w_i \geq 0$$

$$V[i, j] = V[i-1, j] \quad \text{if } j-w_i < 0$$

With the base cases (initial conditions) being:

$V[0, j] = 0$ and
$V[i, 0] = 0$

Pseudocode

Algorithm *dynamicKnapsack(Weight[1..n], Val[1..n], W)*

// **Input**: An array of Weights (Weight) and of values (V) for n items, and a knapsack capacity W

// **Output**: Maximum total value of items that can fit into the knapsack.

// Declare a 2-dimensional (nxW) array for the table

F(0..n, 0..W)

// Initialize the table with base case values.

for i ← 0 **to** n **do**

    F(i, 0) ← 0

for j ← 0 **to** W **do**

    F(0, j) ← 0

// From here is the realization of the recurrence relation

for i ← 1 **to** n **do**

    for j ← 1 **to** W **do**

        if (j – Weight[i]) < 0 **then**

            F(i,j) ← F(i-1, j)

        else

            F(i, j) ← *max* (F(i-1, j), Val(i)+F(i-1, j-Weight[i])

return F(n, W)

	0	1	2	…….	n
0	0	0	0	0	0
1	0				
2	0				
…….	0				
W	0				Sol.

Size of the problem is defined by two variables: n and W

Basic Operation: maximum comparison inside the innermost for loop.

Time Complexity: ***O(nW)*** due to the two nested loops.

Space Complexity: ***O(nW)*** due to the defined two dimensional array.